

Curso: Optimización - Sesión: 14
TEMA 9: ALGORITMOS DE ENJAMBRE - PARTE II

Prof. Marlon Ernesto Moscoso Martínez

UNIVERSIDAD INTERNACIONAL DE LA RIOJA



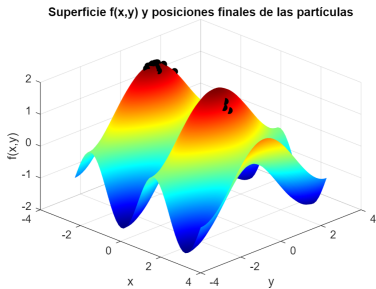
Optimización PER11087

2024 - 2025

email: marlonernesto.moscoso-externo@unir.net

PROBLEMA

¿Cómo emplear el algoritmo de luciérnagas en la localización de extremos locales?



↓¹₉ Contenido

1 Algoritmos de Enjambre de Partículas - PSO

2 Algoritmos de Abejas Artificiales - ABC

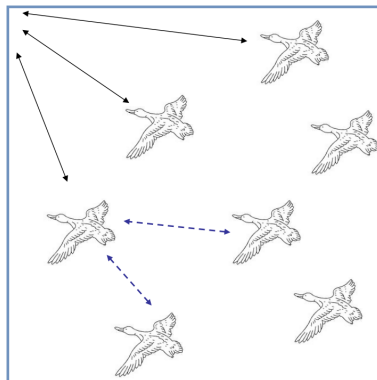
3 Algoritmos de Enjambre de Luciérnagas - FA



- Los **algoritmos de enjambre de partículas** es una técnica de cálculo evolutivo utilizada en optimización.
- Está inspirado en el comportamiento social de grupos en la naturaleza, como bandadas de aves y cardúmenes de peces.
- Los algoritmos de enjambre de partículas fueron introducidos por **Kennedy** y **Eberhart** en 1995.
- En una bandada de pájaros existe un líder que guía a la bandada y el resto de individuos se mueve siguiendo las instrucciones de dicho líder.
- Además, se establecen diferentes comportamientos alrededor de los diferentes individuos para mejorar la exploración y detección de mínimos locales.

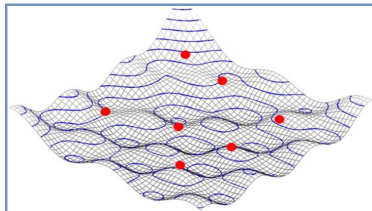
- Los individuos interactúan unos con otros, mientras aprenden de sus propia experiencia.

- Poco a poco los miembros de la población pasan a mejores regiones del problema espacio.



En PSO, una colección de partículas (llamada “enjambre”) se mueven en el espacio de búsqueda buscando la mejor solución a un problema de optimización.

❶ Su **velocidad** se ve afectada tanto por la **posición** de la partícula con la mejor aptitud (**mejor fitness global**), como por la mejor forma aptitud propia de cada partícula (**mejor fitness local**).



❷ La **aptitud (Fitness)** también se refiere a qué tan bien se desempeña una partícula: en una bandada de pájaros, esto podría ser qué tan cerca está un pájaro de una **fuentes de alimento**. En un algoritmo de optimización esto se refiere a la **proximidad** de la partícula al global óptimo.

Basado en una bandada de pájaros: “Existe un líder”.

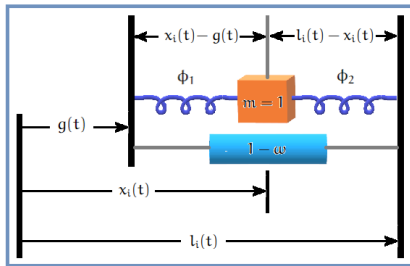
■ La **separación**, es el comportamiento por el que se evita saturar un mismo punto que concentre todos los individuos del enjambre. Esto permite realizar una exploración más extensa del sistema.

■ La **alineación**, permite que todos los individuos se muevan en la dirección promedio. Esto provoca que si hay una dirección favorecida para obtener óptimo, todos los puntos tiendan a seguirla.

■ La **cohesión**, es el comportamiento que hace que los individuos se muevan hacia un mismo punto. Actúa de forma contraria a la separación y permite concentrar los individuos hacia las posiciones más prometedoras.

Deducción del Algoritmo CC-PSO

El algoritmo CC-PSO, puede interpretarse físicamente como un sistema de masa-resorte amortiguado a dos muelles (Fernández - García 2008).



Considerando las constantes de restitución de cada resorte como ϕ_1 y ϕ_2 .

Entonces,

- La partícula en movimiento es un objeto de masa $m = 1$, y el amortiguamiento dado por $1 - \omega$.
- La posición de equilibrio para la i -ésima partícula es $l_i(t)$ y la posición global de equilibrio está dada por $g(t)$.
- Usando el hecho de que actúan la **Ley de Hooke** y que la fuerza de restitución dada por el amortiguamiento, es proporcional a la velocidad de la partícula, el modelo para cada partícula del sistema está dado por:

$$x_i''(t) + (1 - \omega)x_i'(t) + (\phi_1 + \phi_2)x_i(t) = \phi_1 g(t) + \phi_2 l_i(t)$$

En efecto, partiendo del algoritmo en su $k + 1$ - ésima iteración, sabemos que:

- Cada partícula depende de su velocidad y de la aceleración local, pero además de la aceleración global, esto en el sentido del posicionamiento del líder que en este caso puede variar constantemente.
- En este sentido, para $\omega \in \mathbb{R}$ y $|\omega| < 1$ se obtiene que:

$$\begin{cases} v_i^{k+1} = \omega v_i^k + \phi_1(g^k - x_i^k) + \phi_2(l^k - x_i^k) \\ x_i^{k+1} = x_i^k + v_i^{k+1} \end{cases}$$

representan las ecuaciones de actualización tanto de la velocidad como de la posición de la i - ésima partícula en la k - ésima iteración.

Tenemos entonces que:

- 1 ω es el peso de inercia.
- 2 Se definen las constantes $a_l, a_g \in \mathbb{R}$ como las **constantes de aceleración local y global** respectivamente.
- 3 Al discretizar, se obtiene un sistema de ecuaciones en diferencias estocástico.
- 4 En consecuencia, las constantes de aceleración local y global estocásticas verifican:

$$\phi_1 = r_1 a_g$$

y

$$\phi_2 = r_2 a_l$$

- 5 r_1 y r_2 son variables aleatorias uniformemente distribuidas en $(0, 1)$ independientes para cada iteración. En consecuencia, ϕ_1, ϕ_2 son las aceleraciones aleatorias global y local.

- Observe que de la segunda ecuación de actualización, se tiene que: $v_i^{k+1} = x_i^{k+1} - x_i^k$.

- Al sustituir esta expresión en la primera ecuación, queda que:

$$x_i^{k+1} - x_i^k = \omega[x_i^k - x_i^{k-1}] + \phi_1(g^k - x_i^k) + \phi_2(l^k - x_i^k)$$

- Por tanto,

$$x_i^{k+1} + (\phi_1 + \phi_2 - 1 - \omega)x_i^k + \omega x_i^{k-1} = \phi_1 g^k + \phi_2 l^k$$

lo que implica que:

$$x_i^{k+1} + (\phi - 1 - \omega)x_i^k + \omega x_i^{k-1} = \phi_1 g^k + \phi_2 l^k$$

donde $\phi = \phi_1 + \phi_2$ es una variable aleatoria con media

$$\bar{\phi} = \frac{a_l + a_g}{2}.$$

Considerando las condiciones iniciales de cada partícula, es decir la posición inicial y la velocidad inicial de la i - ésima partícula x_{i0} , v_{i0} y teniendo en cuenta que la posición de la i - ésima partícula en la primera iteración depende directamente de estas dos, queda que el algoritmo PSO, está dado por:

Algoritmo PSO

$$\begin{cases} x_i^{k+1} + (\phi - 1 - \omega)x_i^k + \omega x_i^{k-1} = \phi_1 g^k + \phi_2 l^k \\ x_i^0 = x_{i0} \\ x_i^1 = f(x_{i0}, v_{i0}) \end{cases}$$

para alguna relación f . Escogiendo adecuadamente los parámetros (ω, a_l, a_g) para que el punto $(\omega, \bar{\phi})$ pertenezca a la región de estabilidad de primer orden.

A saber, la región:

$$S_D = \{(\omega, \bar{\phi}) : |\omega| < 1, 0 < \bar{\phi} < 2(1 + \omega)\}$$

determina que las trayectorias de las partículas oscilarán alrededor de sus respectivos centros de atracción, a saber

$$o_i^k = \frac{a_g g^k + a_l l^k}{a_g + a_l}$$

lo que establece directamente las condiciones de convergencia del algoritmo. Con la idea de mantener el balance entre la exploración y la explotación de los algoritmos, observamos que se pueden dar versiones del PSO con mayor tendencia a la exploración o con mayor capacidad de convergencia, que es justamente el caso del CC-PSO.

Volvamos a la notación del Material...

La búsqueda del óptimo por enjambre de partículas se basa en las fórmulas iterativas:

$$\begin{cases} v_{id}^{t+1} = v_{id}^t + c_1 R_1(p_{id}^t - x_{id}^t) + c_2 R_2(p_{gd}^t - x_{id}^t) & \text{---> 1} \\ x_{id}^{t+1} = x_{id}^t + v_{id}^{t+1} & \text{---> 2} \end{cases}$$

- v_{id}^t es la velocidad de una partícula dentro del enjambre y x_{id}^t es su posición.
- La i , es el índice de la partícula y la d indica la componente dentro del espacio de búsqueda,

Por otra parte:

- t es la época o iteración del método.
- c_1 y c_2 son parámetros de control que permiten regular la afectación de la mejor posición detectada alrededor de los distintos individuos.
- p_i es la mejor posición encontrada por la i - ésima partícula y p_g es la mejor posición encontrada por el total de las partículas.
- Los valores R_1 y R_2 son valores aleatorios generados mediante una variable aleatoria uniforme entre 0 y 1.

Estructura del PSO - Maximización

- ① Inicializamos n partículas del sistema. Para ello:
 - Se inicializa cada componente de la posición mediante una variable aleatoria uniforme entre el límite inferior $\mathbf{b}_{ld} = (b_{l1}, \dots, b_{lm})$ y el superior $\mathbf{b}_{ud} = (b_{u1}, \dots, b_{um})$ del espacio de búsqueda, $x_{id}^0 = U(\mathbf{b}_{ld}, \mathbf{b}_{ud})$.
 - Inicializamos la mejor posición de la partícula al mismo valor $p_i = x_i^t$.
 - Si $f(p_i) > f(p_g)$, entonces definimos $p_g = p_i$.
 - Inicializamos la velocidad de cada partícula como una variable uniforme, $v_{id} = U(-|\mathbf{b}_{ud} - \mathbf{b}_{ld}|, |\mathbf{b}_{ud} - \mathbf{b}_{ld}|)$.

- ② Mientras no se cumpla el criterio de parada:
 - Para cada partícula empleamos las ecuaciones ① y ② para actualizar su posición y velocidad.
 - Si $f(x_i) > f(p_i)$, entonces $p_i = x_i$.
 - Si $f(p_i) > f(p_g)$, entonces $p_g = p_i$.
- ③ El óptimo local encontrado viene dado por p_g .

Ejemplo de PSO

Encuentra el máximo de la función:

$$f(x, y) = \sin(2x) + \cos(y)$$

empleando el algoritmo de enjambre de partículas.

Se resolverá con la implementación del algoritmo:

Paso 1 Establecer Función Objetivo

$$f = \sin(2x) + \cos(y);$$

Paso 2 Inicializar

```
npar = 100;
```

```
% inicializamos particulas y velocidades
```

```
x = -pi + rand([npar, 2]) * 2 * pi;
```

```
v = (-2 * pi + rand([npar, 2]) * 4 * pi) * 0.1;
```

Paso 3 Inicializar parámetros

```
% inicializamos parámetros
```

```
c1=0.1;
```

```
c2=0.2;
```

Paso 4 Bucle principal

```
it=0;
```

```
while(it <15)
```

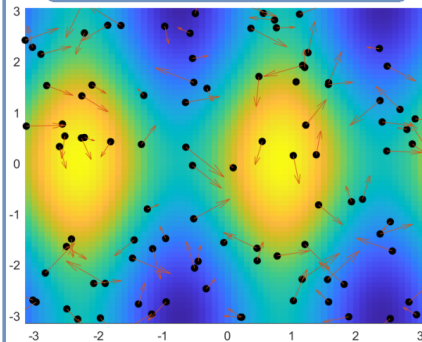
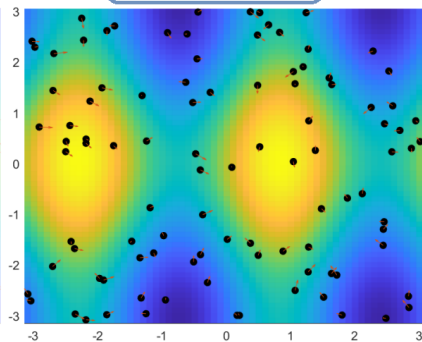
```
v=v+ c1 * rand (npar ,2) .*( xopt -x) + ... c2* rand  
(npar ,2) .* ( xoptg. * ones ( size (x))-x);
```

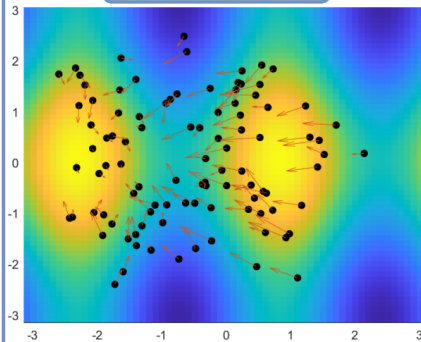
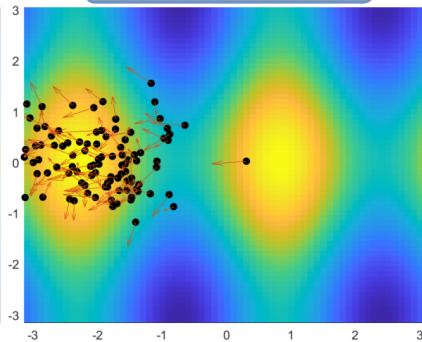
```
x=x+0.2*v;
```

```
⋮
```

 **Nota** Cuando ejecutamos el código anterior, obtenemos como resultado el valor $(x, y) = (-2.3562, 0.0148)$ y el valor de la función objetivo $f(x,y) = 1.9999$.

	foptg	1.9996
	ioptg	60
	it	15
	npar	100
	xoptg	[-2.3488,0.0235]

Distribución Inicial de Partículas $it = 0$ Quinto iterado $it = 5$ 

Décimo iterado $it = 10$ Décimo quinto iterado $it = 15$ 

Mejoras en el Algoritmo

Con el paso del tiempo se han realizado varias exploraciones y seguido varias ideas para mejorar el algoritmo:

- La principal modificación se debe a Shi y Eberhart (Shi and Eberhart, 1998) en la que se añade un factor ω a la variación de la velocidad, de forma que la fórmula ❶ resulte en:

$$v_{id}^{t+1} = \omega v_{id}^t + c_1 R_1(p_{id}^t - x_{id}^t) + c_2 R_2(p_{gd}^t - x_{id}^t) \rightarrow \text{❸}$$

- Cuando ω toma un valor grande, la velocidad se incrementa dando una mayor importancia a la exploración del sistema y facilitando la búsqueda en el mismo.
- Mientras que si el valor es pequeño, se tiende hacia la cohesión y uniformización de todos los individuos.

- ω se denomina **peso de inercia** al parámetro.
- La fórmula dada en la ecuación ③, se considera la fórmula estándar para este tipo de problemas.
- En ocasiones, las velocidades de búsqueda pueden ser muy grandes, esto provoca que las partículas escapen de la zona de búsqueda. Para evitar este comportamiento Clerc y Kennedy introdujeron un término limitante K para aumentar las probabilidades de convergencia (Clerc and Kennedy, 2002) de forma que la Ecuación ① queda como:

$$v_{id}^{t+1} = K \left[v_{id}^t + c_1 R_1 (p_{id}^t - x_{id}^t) + c_2 R_2 (p_{gd}^t - x_{id}^t) \right]$$



Los algoritmos de abejas artificiales se basan, como su nombre indica, en la estrategia de exploración, búsqueda y recolección del néctar por parte de las abejas. Fue desarrollado por Karaboga en 2005.

El algoritmo establece una analogía entre la función objetivo y la comida o néctar, divide a los individuos en 3 tipos de abejas:

- Las **abejas empleadas** se dedican a la búsqueda de nueva comida en las zonas de búsqueda que tienen asignadas.
- Las **abejas observadoras** acuden a las zonas de búsqueda para evaluar la cantidad de comida en dicha zona.
- Las **abejas exploradoras** buscan nuevas fuentes de comida

Inicialización

Supongamos que queremos optimizar una función en un espacio de búsqueda real $\mathbb{R}^D$. Entonces:

- Definiremos una malla inicial de soluciones factibles (**fuentes de comida**).
- Llamamos x_{id} al conjunto de soluciones. Aquí, $i = 1, \dots, M$ es el tamaño de la malla y $d \in \{1, \dots, D\}$ recorre las componentes de cada solución factible.
- Las fuentes de comida son detectadas por las exploradoras, según el valor:

$$x_{id}^0 = b_{ld} + R(b_{ud} - b_{ld})$$

Donde b_l y b_u son los límites inferior y superior del espacio de búsqueda.

Estructura del Algoritmo

❶ **Fase de las abejas empleadas.** Cada abeja empleada intenta mejorar la una solución factible. Para ello, consideramos:

$$v_{id} = x_{id} + R_e(x_{id} - x_{kd})$$

Donde, R_e es un valor aleatorio en el rango $[-a, a]$ definido como parámetro de control y k es un índice que se selecciona de forma aleatoria entre las fuentes de comida.

❷ **Fitness.** Utilizamos una función de fitness F , para ver si v_i es mejor que x_i . Si $F(v_i) < F(x_i)$, entonces $x_i = v_i$.

La función F , está dada por:

$$F(\mathbf{x}) = \begin{cases} \frac{1}{1 + f(\mathbf{x})} & , \quad \text{si } f(\mathbf{x}) \geq 0 \\ 1 + |f(\mathbf{x})| & , \quad \text{si } f(\mathbf{x}) \leq m \end{cases}$$

③ Fase de las abejas observadoras. Las abejas observadoras seleccionan una fuente de alimentos de forma aleatoria de entre todas las disponibles dependiendo del valor de de la función de fitness asignada a la fuente. Para cada fuente se fija una probabilidad:

$$p_i = \frac{F(x_i)}{\sum_{j=1}^n F(x_j)}$$

④ Se aplica la mejora otra vez.

⑤ **Fase de las abejas exploradoras.** Después de un número de iterados N en los que una abeja no consigue mejorar el valor de la función objetivo, esta se transforma en exploradora. Una abeja exploradora seleccionará una fuente de comida dentro del espacio de búsqueda igual que se ha realizado en la fase de inicialización.

 **Nota:** Este tipo de algoritmo suele ser sencillo de implementar y es robusto y flexible.

- No depende de las derivadas de la función objetivo.
- Deberemos fijar la región de búsqueda, así como el rango de búsqueda alrededor de una fuente de comida a , el tamaño de la colonia de abejas y el número de iterados N que debemos aplicar para que una abeja empleada pase a ser exploradora.

Artículos de interés

XIII CONGRESO ESPAÑOL EN METAHEURÍSTICAS Y ALGORITMOS EVOLUTIVOS Y BIOINSPIRADOS



Algoritmo Multiobjetivo de Colonia de Abejas Artificiales aplicado al Problema de Orientación

Rodrigo Martín-Moreno

Depto. Tecnología de Computadores y Comunicaciones
Universidad de Extremadura
 Cáceres, España
 rmartinky@alumnos.unex.es

Miguel A. Vega-Rodríguez

Depto. Tecnología de Computadores y Comunicaciones
Universidad de Extremadura
 Cáceres, España
 mavega@unex.es

Algoritmo 1 Colonia de Abejas Artificiales Multiobjetivo

entrada: T_{max} (tiempo máximo de ejecución), TP (tamaño de la población) y $limite$ (criterio de abandono)

salida: $Fichero_SND$ (fichero con las soluciones no dominadas)

```

1:  $t \leftarrow 0$ ;  $Fichero\_SND \leftarrow \emptyset$ 
2: IniciarFuentesAlimento( $TP$ )                                ▷ ver IV-C
3: while  $t < T_{max}$  do
4:   FaseAbejasObreras( $TP$ )                                    ▷ ver IV-D
5:   CalcularCantidadNectar( $TP$ )                                ▷ ver IV-E
6:   FaseAbejasObservadoras( $TP$ )                                ▷ ver IV-F
7:   FaseAbejasExploradoras( $TP$ ,  $limite$ )                        ▷ ver IV-G
8:   GuardarSND( $Fichero\_SND$ )                                   ▷ ver IV-H
9: end while
```

maximizar $F(x) = (f_1(x), \dots, f_K(x))$,

$$f_k = \sum_{v_i \in V \setminus \{v_0, v_{n+1}\}} (b_{ik} \cdot y_i) \quad (k = 1, \dots, K),$$

2016

Artículos de interés

REVISTA DE MATEMÁTICA: TEORÍA Y APLICACIONES 2014 **21**(1) : 107–126

CIMPA – UCR ISSN: 1409-2433

COLONIA DE ABEJAS ARTIFICIALES Y OPTIMIZACIÓN POR ENJAMBRE DE PARTÍCULAS PARA LA ESTIMACIÓN DE PARÁMETROS DE REGRESIÓN NO LINEAL

ARTIFICIAL BEE COLONY AND PARTICLE SWARM OPTIMIZATION FOR THE ESTIMATION OF NONLINEAR REGRESSION PARAMETERS

Abstract

This paper shows the comparison results of ABC (Artificial Bee Colony) and PSO (Particle Swarm Optimization) heuristic techniques that were used to estimate parameters for nonlinear regression models. The algorithms were tested on 27 data bases from the NIST collection (2001), 8 of these are considered to have high difficulty, 11 medium difficulty and 8 low difficulty. Experimental results are presented.

SERGIO DE-LOS-COBOS-SILVA*

MIGUEL A. GUTIÉRREZ-ANDRADE†

ERIC A. RINCÓN-GARCÍA‡ PEDRO LARA-VELÁZQUEZ§

MANUEL AGUILAR-CORNEJO¶

*Universidad Autónoma Metropolitana-Iztapalapa, Departamento de Ingeniería Eléctrica, Av. San Rafael Atlixco 186, Col. Vicentina, Del. Iztapalapa, México D.F., C.P. 09340, México. E-Mail: cobos@xanum.uam.mx

†Misma dirección que/Same address as: S. de-los-Cobos-Silva. E-Mail: gamma@xanum.uam.mx

‡Universidad Autónoma Metropolitana-Azcapotzalco, Departamento de Sistemas, Av. San Pablo 180, Colonia Reynosa Tamaulipas, México D.F., C.P. 02200, México. E-Mail: rigaeral@correo.azc.uam.mx

§Misma dirección que/Same address as: E. Rincón-García. E-Mail: plara@xanum.uam.mx

¶Misma dirección que/Same address as: S. de-los-Cobos-Silva. E-Mail: mac@xanum.uam.mx



El **algoritmo de enjambres de luciérnagas** fue desarrollado por Krihnanand y Ghose en 2009. En este tipo de algoritmos:

- Los individuos que juegan un papel importante son luciérnagas.
- La idea fundamental será que una luciérnaga se acercará hacia aquella compañera que tenga un valor más elevado de intensidad lumínica de entre las que están a su alrededor.

Parámetros de cada Luciérnaga

- 1 **Posición** ($x_i(t)$): la posición dentro del espacio de búsqueda de nuestra función.
- 2 **Nivel de luciferina** ($l_i(t)$): representa el nivel de brillo de la luciérnaga. Será el equivalente con memoria de la función objetivo.
- 3 **Radio de búsqueda** ($r_i(t)$): es el radio en el que una luciérnaga es capaz de ver a sus compañeros. Si una luciérnaga está más lejos del radio de búsqueda, se considerará que no existe.

Estructura del Algoritmo

Se inicializa el algoritmo con una distribución aleatoria de luciérnagas.

❶ **Actualizamos el nivel de luciferina.** En esta fase se evalúa el nivel de luciferina de cada luciérnaga.

Para ello, empleamos la fórmula:

$$l_i(t) = (1 - \alpha) \cdot l_i(t - 1) + \gamma f(x_i(t))$$

Donde α es el nivel de disipación de la luciferina, γ es un parámetro de control y f la función objetivo.

② Actualizamos la posición de las luciérnagas acorde a la función:

$$x_i(t) = x_i(t-1) + s \left(\frac{x_k(t-1) - x_i(t-1)}{\|x_k(t-1) - x_i(t-1)\|} \right)$$

Donde s es la longitud del paso, k es la luciérnaga hacia la que se mueve la luciérnaga i .

¿Cómo seleccionamos k ? Usamos:

$$p_{i,k}(t) = \frac{l_k(t) - l_i(t)}{\sum_{j \in B_i(t)} l_j - l_i}$$

donde $p_{i,k}(t)$ nos indica la probabilidad que la luciérnaga i se mueva en la dirección de la luciérnaga k y B_i es el conjunto de luciérnagas j tales que $\|x_j - x_i\| < r_i(t)$ y $l_j > l_i$.

③ Actualizamos el rango de búsqueda.

- Usaremos la fórmula:

$$r_i(t+1) = \min\{r_s, \max\{0, r_i(t) + \beta(n_d - n_i(t))\}\}$$

- a** r_s es el rango máximo para una luciérnaga.
- b** n_d es el número de vecinos deseado.
- c** $n_i(t)$ es el número de luciérnagas en el radio de observación del la i -ésima luciérnaga.
- d** β es un parámetro de control para modificar el radio de observación.

Parámetros de Control

- a** El parámetro de difusión de luciferina α .
- b** El parámetro de peso de la función objetivo γ .
- c** La longitud de avance de paso de la luciérnaga s .
- d** El rango máximo de la luciérnaga r_s .
- e** El número de vecinos en el rango de observación deseado n_d .
- f** El parámetro de control del radio de observación β .

 **Nota:** La elección de estos parámetros siempre dependerá de la experiencia del diseñador y del problema que estemos realizando.

Ejemplo de Enjambre de Luciérnagas

Encuentra el máximo de la función:

$$f(x, y) = \sin(2x) + \cos(y)$$

empleando el algoritmo de enjambre de luciérnagas.

Se resolverá con la implementación del algoritmo:

Paso 1 Establecer Función Objetivo

$$f = \sin(2x) + \cos(y);$$

Paso 2 Inicializar % Número de luciérnagas

```
nluc = 100;
```

```
% inicializamos particulas y luminancia
```

```
x = -pi + rand([nluc, 2]) * 2 * pi;
```

```
l = zeros([nluc, 1]);
```

Paso 3 Inicializar parámetros

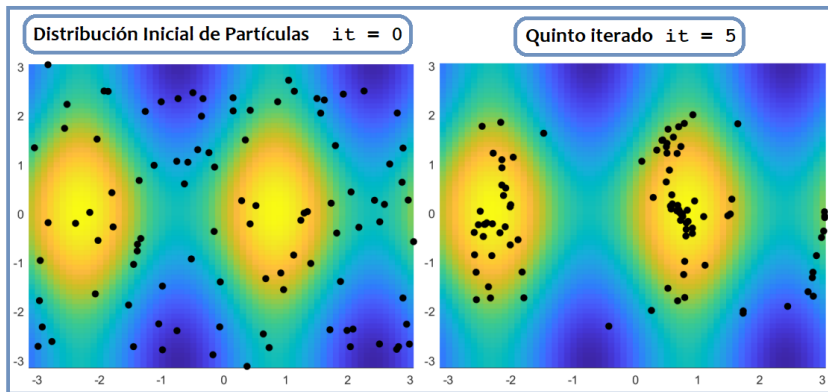
```
% vector inicial para el cálculo
idvec =0: nluc -1;
% inicializamos parámetros
beta =0.1;nd =4;alpha =0.5;gamma =0 .7;s=0 .3;
% visión inicial de las luciérnagas
r= ones ([ nluc ,1]) *rs /2;
```

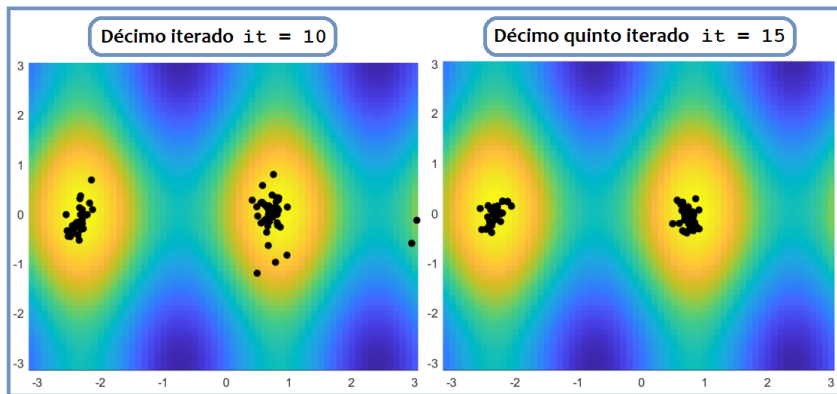
Paso 4 Bucle principal

```
% actualización de la luciferina
l=(1 - alpha )*l+ gamma *f(x (: ,1) ,x (: ,2) );
```



Nota Cuando ejecutamos el código anterior, obtenemos como resultado el valor $(x,y) = (0.7877, -0.027)$ y el valor de la función objetivo $f(x,y) = 1.9996$.





UNIVERSIDAD INTERNACIONAL DE LA RIOJA

